

SQL



→ SQL stands for Structured Query Language.

→ SQL is Relational Database Management System to handle structured data.

Database

@programmingwithrathod

Database is collection of data in a format that can be easily accessed

A software application used to manage our DB is called DBMS [Database management system]

Types of Database

Relational (RDBMS)

- Data stored in tables

Example: MySQL, ORACLE, SQL server Microsoft

Non - relational (NoSQL)

- data not stored in table

Example : MongoDB

What is SQL ?

SQL is a programming Language used to ~~interact~~ interact with relational database.

It is used to perform CRUD operations :

C = Create

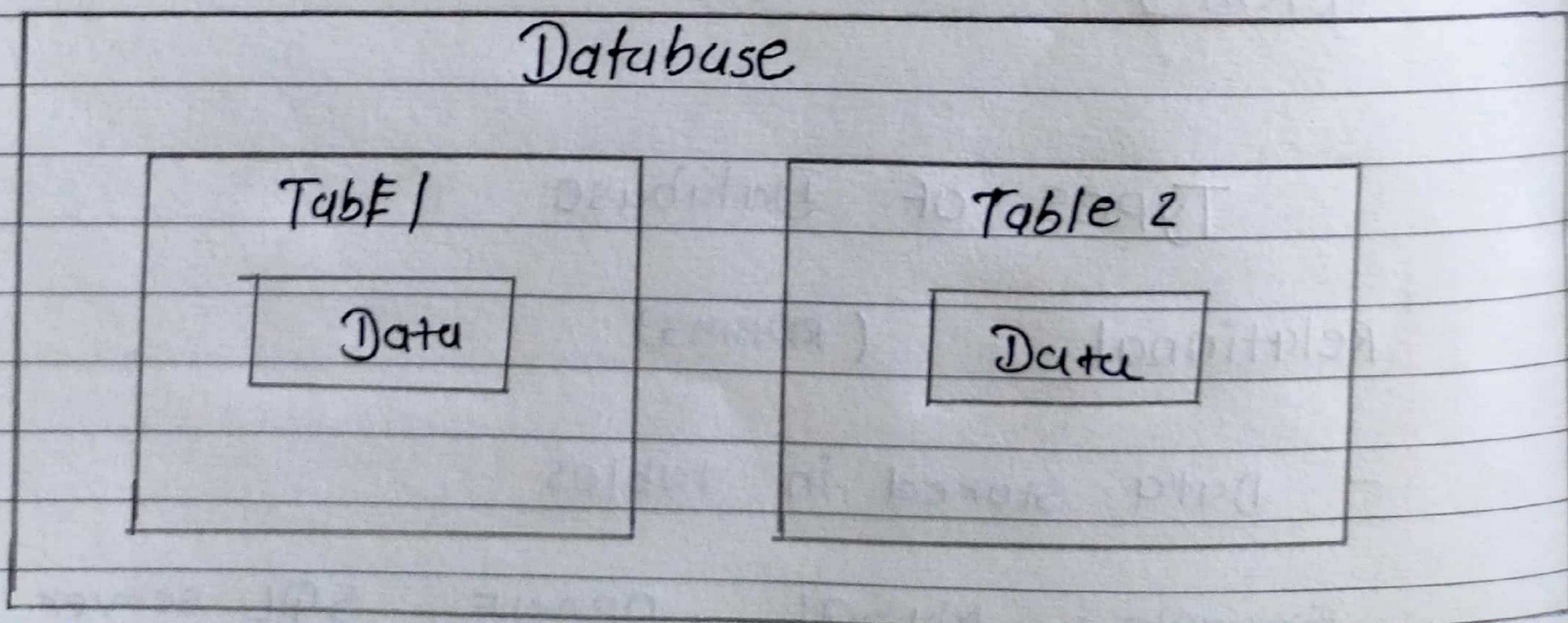
R = Read

U = Update

D = Delete

@programmingwithrathod

Database Structure



What is a table ?

table is a rows and columns combination.

Creating our First Database

- Our first SQL Query

creating Database Db-name; ↘ Full stop

Drop Database Db-name; ↘ Delet Database

Example :

CREAT DATABASE college; ↘ Database - Name

DROP DATABASE college;

@programmingwithrathod

- USE Database query

USE Database_name ;

The use command is used when there are multiple database in the sql and the user or programmer specifically wants to use a particular database.

Example :

USE college ;

Creating our First table

- **Create Table** table name (
 column_name1 datatype constraint,
 column_name2 datatype constraint,
);

Example :

```
CREATE TABLE student (  
    id INT primary key,  
    sname varchar (50),  
);
```

@programmingvibrathod

id	sname
.....	
.....	
.....	

- **INSERT INTO Query**

The INSERT INTO statement is used to insert new record in a table.

SQL Datatypes

They define the type of values that can be stored in a column.

Datatype

CHAR

VARCHAR

BLOB

INT

TINYINT

BIGINT

BIT

FLOAT

BOOLEAN

DATE

YEAR

@programmingwithrathod

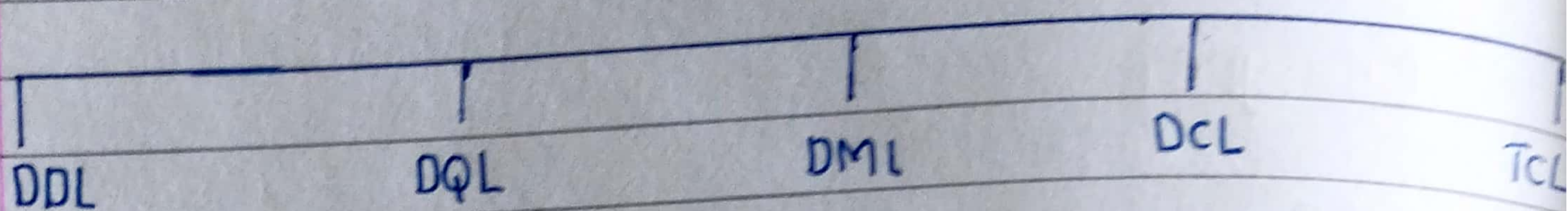
Signed & Unsigned

TINYINT UNSIGNED (0 to 255)

TINYINT (-128 to 127)

- unsigned can store only zero and positive numbers.
- Signed allow zero, positive and negative number.

Types of SQL Commands



DDL : (Data Definition Language) : Create, alter, rename, truncate & drop

DQL (Data Query Language) : select

DML (Data manipulation Language) : Insert, update & delete

DCL (Data control Language) : grant & revoke permission to user

TCL (Transaction control Language) : start transaction, commit, rollback &

@programmingwithme

Database related Query

CREATE DATABASE db-name;

CREATE DATABASE IF NOT EXISTS db-name;

- SHOW Database;

- Show Tables;

Table related command's

• ~~SQL~~ CREATE

```
CREATE TABLE student (  
  id INT primary key,  
  name varchar (50),  
);
```

• Select & view All columns

```
SELECT * FROM table-name;
```

• Insert

@programmingwithrathod

```
INSERT INTO table-name  
( column 1, column 2 )  
values  
( col1-v1, col2-v1 ),  
( col1-v2, col2-v2 );
```

• DROP (Delete Table)

```
DROP TABLE table-name;
```

- It's statement is used to drop an existing table in a database.

Example:

```
DROP TABLE student;
```


practice Qs 1

Q.5 Create a database for your company named programming.

Step 1 : create a table inside this DB to store student info [id, name and age].

Step 2 : Add following information in the DB:

1, "vijay", 17
2, "syam", 18
3, "ajay", 15

Step 3 : select & view all your table data.

@programmingwithrathod

CREATE DATABASE programming ;
USE programming ;

CREATE TABLE student (
 id INT primary key ,
 name VARCHAR (100),
 age salary INT
);

INSERT INTO student
(id , name , age)
VALUES
(1 , " vijay " , 17),
(2 , " syam " , 18),

(3, "AJay", 15) ;

SELECT * FROM Student ;

Keys

primary key :

It is a column in a table that uniquely identifies each row . (a unique id)

There is only 1 primary key & it should be NOT null .

@programmingwithrathod

Foreign key

A Foreign key is a column in a table that refers to the primary key

There can be multiple F/K.

FKs can have duplicate & null values.

Example :

table 1 - Student

table 2 - city

primary key

Foreign key

id	cityid	city
102	(1)	pune
101	2	mumbai
103	(1)	pune

primary key

id	city
1	pune
2	mumbai
3	Delhi

Constraints

SQL constraints are used to specify rules for data in a table.

NOT NULL columns cannot have a null value

UNIQUE all values in column are different

primary key makes a column unique & not null but used only for one

foreign key prevent actions that would destroy links between tables

DEFAULT sets the default value of a column

CHECK it can limit the values allowed in a column

Syntax :

@programmingwithrathod

```
create table Name (
    id INT Primary key,
    Name varchar(50),
    age INT,
    constraint age_check CHECK
        (age >= 18 AND Name =
            'RAHUL')
);
```

```
create table new (
    age INT CHECK (age >= 18)
);
```


Select in Detail

used to select any data from the database

Basic Syntax

```
select col1, col2 From table-name ;
```

To select all

```
select * From table-name ;
```

Where clause

@programmingwithrathod

To define some conditions

Syntax :

```
SELECT col1, col2 FROM table-name  
WHERE Condition ;
```

Example :

```
SELECT * From student WHERE marks > 85 ;
```

Using operators in WHERE

Arithmetic OP : +, -, *, /, %

Comparison OP : =, <, >, <=, >=, !=

Logical OP : AND, OR, NOT, IN, BETWEEN, All,

Operators

AND to check for both condition to be true

OR to check for one of the condition to be true

Between select for a given range

(select * from student
where mark between 80 and 90;)

IN

matches any value in the list

(same IN ("Delhi", "Mumbai");)

NOT to negate the given condition

Limit Clause

@programmingwithrathod

sets an upper limit on number of rows to be returned

SELECT * FROM student LIMIT 3;

Order By clause

(ASC)

(DESC)

To sort in ascending or descending order

SELECT * FROM student
order By city ASC;

Aggregate Functions

Aggregate function perform a calculation on a set of value, and return a single value.

- COUNT()
- MAX()
- MIN()
- SUM()
- AVG()

Example : 1) SELECT MAX (marks)
From Student ;

2) SELECT AVG (marks)
From Student ;

@programmingwithrumoel

Group By clause

Groups rows that have the same values into Summary rows.

It collect data from multiple record and groups the result by one or more column.

Example :

```
SELECT city , count (name)
From student
Group By city ;
```


Having clause

Similar to where i.e. ~~application~~ Applies some condition on rows.

used when we want to apply any condition after grouping.

Example :

```
select count (name), city
From student
Group By city
Having max (marks) > 90 ;
```

@programmingwithruhod

General order

```
select columns
From table_name
where condition
Group By columns
Having condition
Order By column(s) ASC
```

Example :

```
select city
FROM student
WHERE grad = " c "
GROUP BY city
HAVING MAX (marks) >= 90
order By city DESC ;
```


Table related Queries

- update (update existing rows)

```
UPDATE table-name  
set col1 = Val1, Val2 = col2  
WHERE condition
```

- Delete (to Delete existing rows)

```
delete From table-name  
where condition ;
```

@programmingwithratn0x1

Cascading for Foreign key

on Update cascading

When we create a Foreign key using update cascade the referencing rows are updated in the child table when the referenced row is updated in the parent table which has a primary key.

on Delete cascade

When we created a Foreign key using this option, it delete the Referencing rows in the child table when the referenced row is deleted in the parent table which has a pk.

- Alter (to change the schema)

ADD column

Alter Table table-name

ADD column column-name datatype;

DROP column

Alter Table table-name

DROP column column-name;

Rename Table

Alter Table table-name;

Rename To new-table-name;

change column (rename)

Alter table table-name

change column old-name new-name;

modify column (modify datatype)

Alter Table table-name

Modify col-name new-datatype;

- Truncate (to delete table's data)

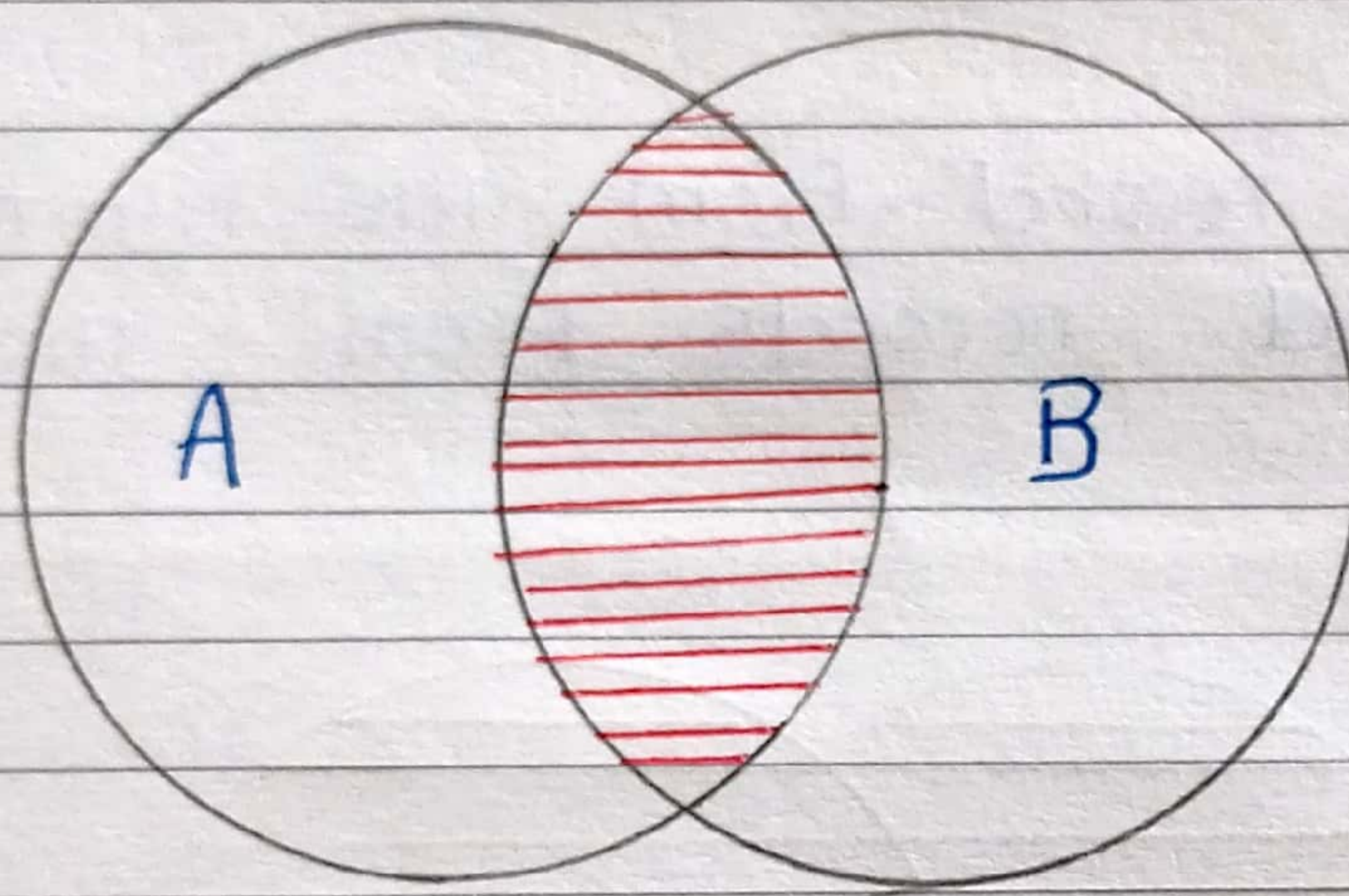
Truncate Table table-name;

@programmingwithrathod

Joins in SQL

Joins is used to combine row from two or more data tables, based on a related column between them.

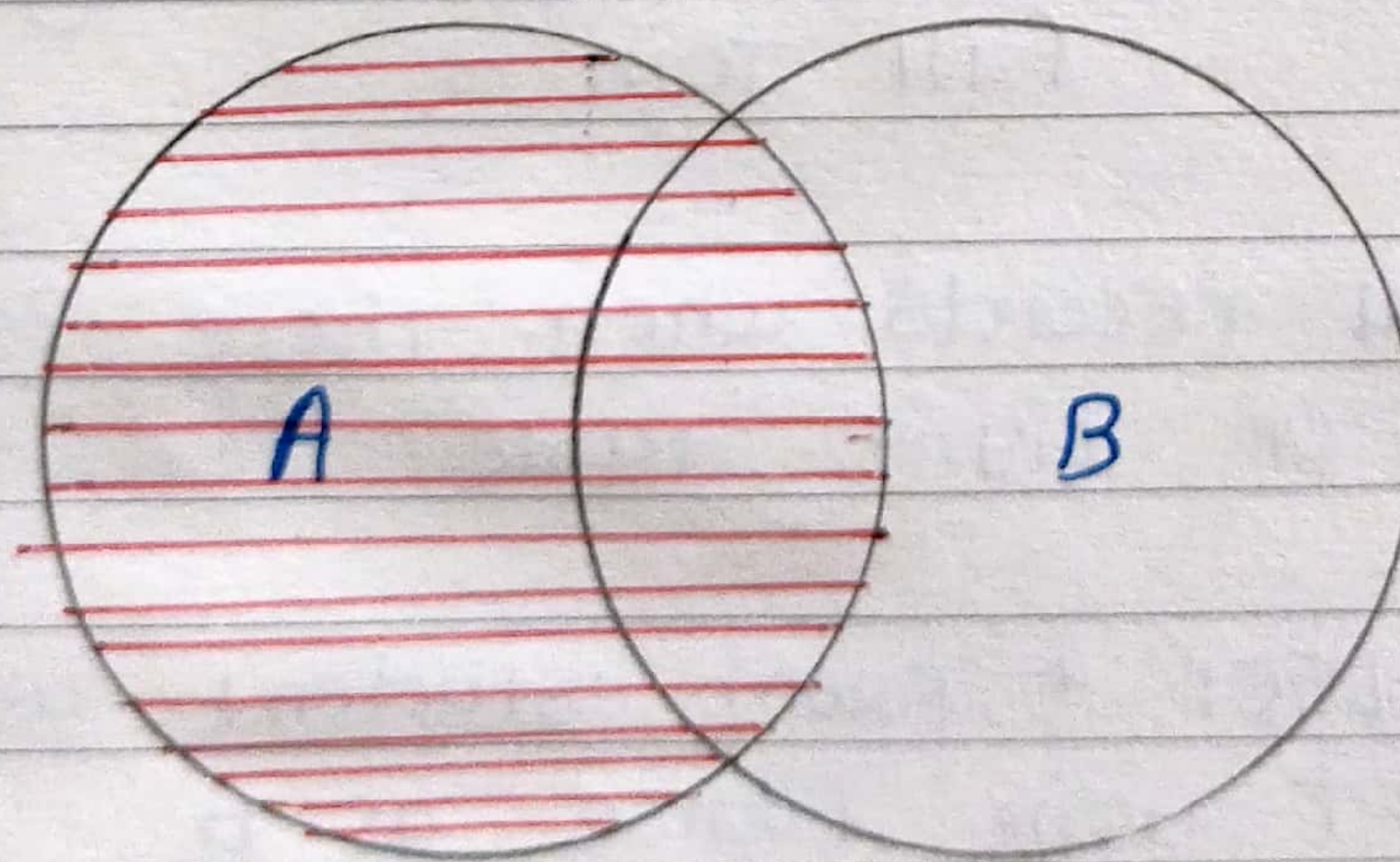
Type's of Joins



Inner Join

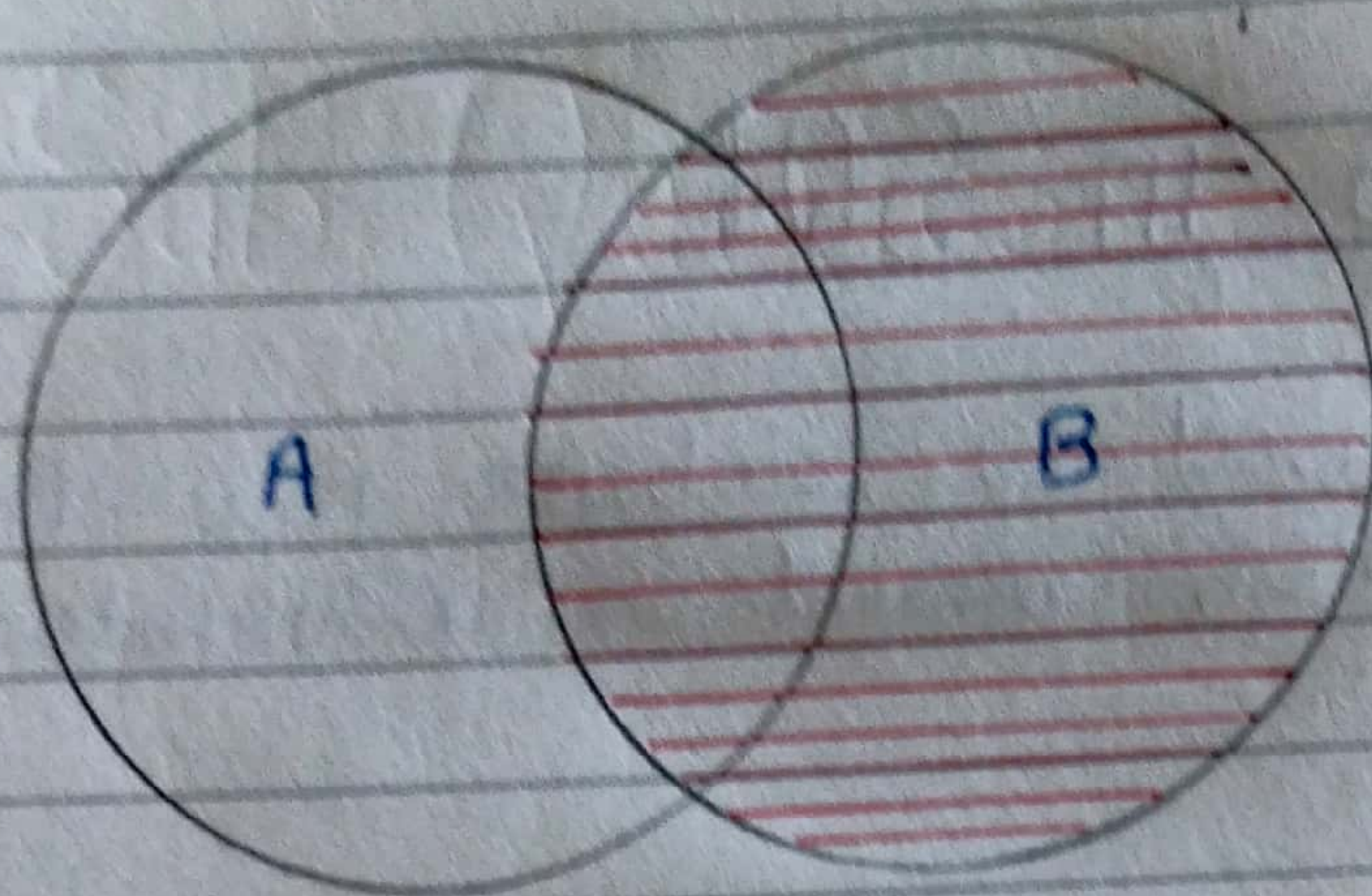
@programmingwithmoel

- Return records that have matching value in both tables.



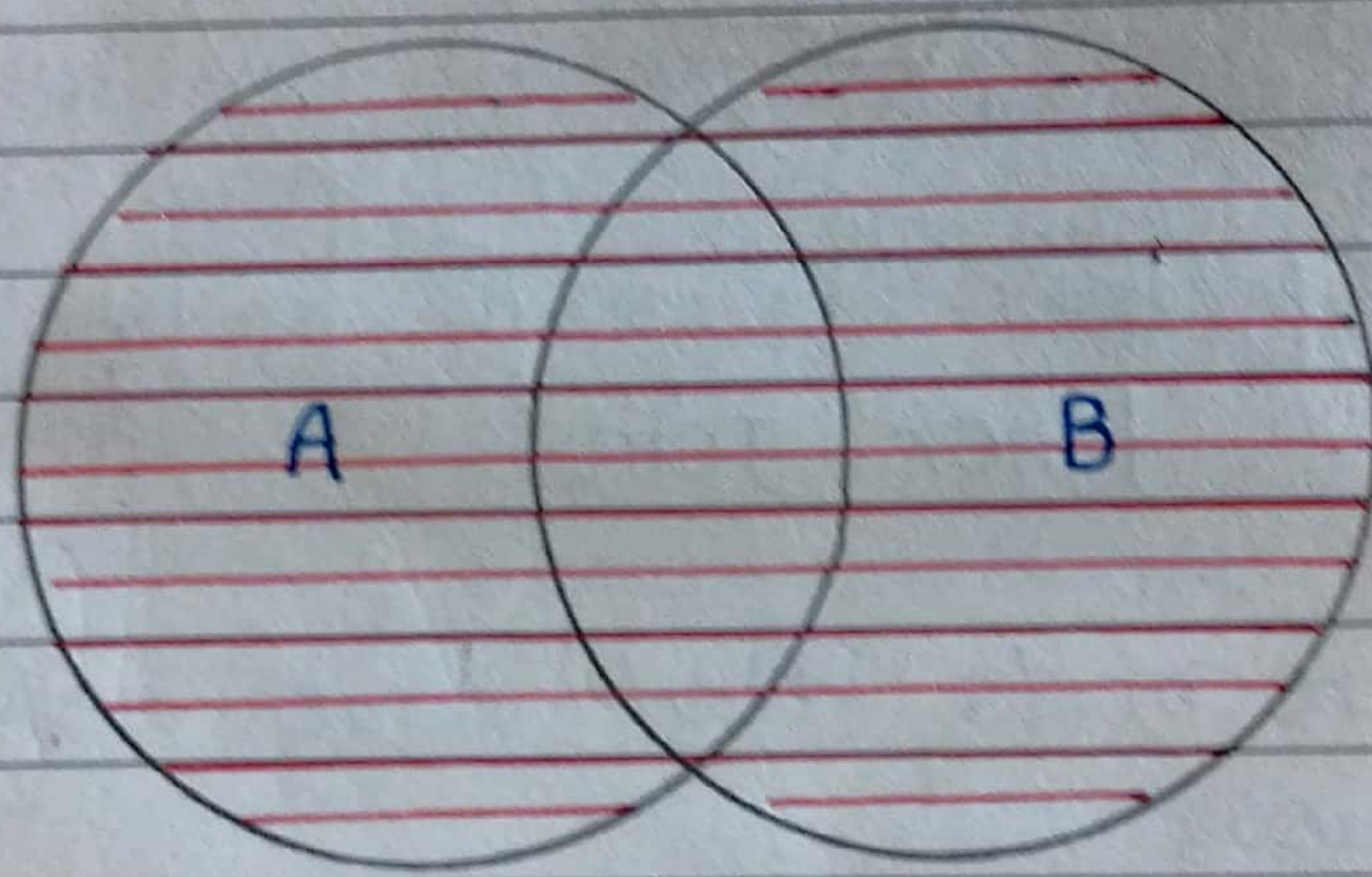
Left Join

- Return all records from the left table, and the matched record from the right table.



Right Join

- Returns all record from the right table, and the matched records from the left table.



Full Join

@programmingwithh

- Returns all records when there is a match in either left or right table

Syntax : `SELECT * FROM student as a
LEFT JOIN course as b
ON a.id = b.id
union`

`SELECT * FROM student as a
RIGHT JOIN course as b
ON a.id = b.id ;`

Left Join
union

Right Join

Self Join

It is a regular join but the table is joined with itself.

Syntax :

```
SELECT column s  
FROM table as a  
JOIN table as b  
ON a.col_name = b.col_name ;
```

Union

@programmingwithrathod

It is used to combine the result-set of two or more SELECT statements.

Given union records.

To use it :

- every select should have same no. of column
- column must have similar data types
- column in every select should be in same order

Syntax :

```
SELECT column(s) FROM table A  
union  
SELECT column(s) FROM table B
```


SQL Sub Queries

A Subquery of Inner query or a Nested query is a query within another SQL query.

It involves 2 select statements.

Syntax :

```
SELECT column(s)
FROM table-name
WHERE col-name operator
(subquery);
```

@programmingwithrathod

MySQL Views

A view is virtual table based on the result-set of an SQL statements.

* A view always shows up-to-date data. The database engine recreates the view, every time a user queries it.

Syntax :

```
CREATE VIEW creedit AS
SELECT Id, name FROM student-bank;

SELECT * FROM creedit
```